

Practical Machine Learning

Hands-on Exercises with PyTorch

Pavanakumar Mohanamurthy

What is PyTorch?

PyTorch is a Python library for:

- ▶ Building neural networks
- ▶ Automatic gradient computation
- ▶ GPU acceleration

Why PyTorch?

- ▶ Easy to learn (Python-like)
- ▶ Used by researchers worldwide
- ▶ Great for prototyping

```
import torch
import torch.nn as nn

# Create a tensor (like
# numpy array)
x = torch.tensor([1.0, 2.0,
                  3.0])

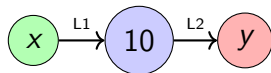
# Basic operations
y = x * 2          # [2, 4, 6]
z = x.sum()        # 6.0
```

Building a Neural Network in PyTorch

```
class MyNetwork(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.layer1 = nn.Linear  
            (1, 10)  
        self.layer2 = nn.Linear  
            (10, 1)  
  
    def forward(self, x):  
        x = torch.tanh(self.  
            layer1(x))  
        x = self.layer2(x)  
        return x
```

Key Components:

- ▶ `nn.Module`: Base class
- ▶ `nn.Linear(in, out)`: Dense layer
- ▶ `forward()`: Data flow
- ▶ `torch.tanh()`: Activation



The Training Loop (3 Simple Steps)

```
for epoch in range(100):  
    # Step 1: Forward pass  
    pred = model(x_data)  
  
    # Step 2: Compute loss  
    loss = loss_fn(pred, y_data)  
    )  
  
    # Step 3: Backward pass  
    optimizer.zero_grad()  
    loss.backward()  
    optimizer.step()
```

Each epoch:

1. **Forward:** Make prediction
2. **Loss:** Measure error
3. **Backward:** Update weights

Forward → Loss → Backward

What is an Epoch?

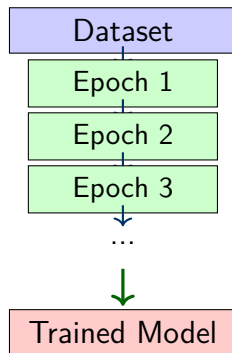
Epoch = One complete pass through the entire training dataset

Example:

- ▶ Dataset has 1000 samples
- ▶ 1 epoch = model sees all 1000 samples once
- ▶ 10 epochs = model sees each sample 10 times

Why multiple epochs?

- ▶ One pass is usually not enough
- ▶ Model learns gradually
- ▶ Each epoch refines the weights



More epochs → better learning
(but too many → overfitting)

What is an Optimizer?

Problem: How do we update weights to reduce error?

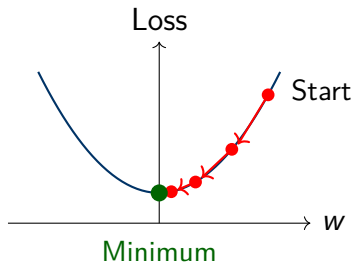
Solution: Use gradient descent!

- ▶ Compute gradient (slope) of loss
- ▶ Move weights in opposite direction
- ▶ Repeat until loss is small

Update Rule:

$$w_{new} = w_{old} - \eta \cdot \frac{\partial \text{Loss}}{\partial w}$$

where η is the **learning rate**



Each step moves toward lower loss

SGD: Stochastic Gradient Descent

The Simplest Optimizer

Update Rule:

$$w = w - \eta \cdot \frac{\partial \text{Loss}}{\partial w}$$

PyTorch:

```
optim.SGD(model.parameters()  
          , lr=0.01)
```

Pros: Simple, low memory

Cons: Slow, sensitive to lr

“Stochastic” = uses mini-batches

Adam: Adaptive Moment Estimation

A Smarter Optimizer

Adapts learning rate per weight:

- ▶ Tracks **momentum**
- ▶ Tracks **velocity**
- ▶ Auto-adjusts step size

PyTorch:

```
optim.Adam(model.parameters  
            (), lr=0.001)
```

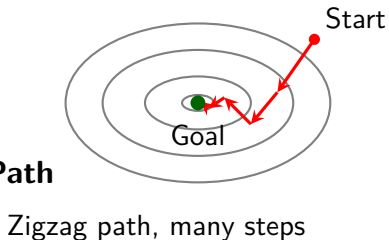
Pros: Fast, works out-of-box

Cons: More memory

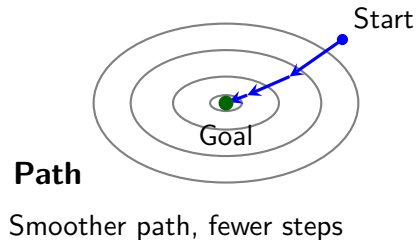
Rule: Start with Adam!

SGD vs Adam: Visual Comparison

SGD Path



Adam Path



Adam uses momentum to avoid zigzag behavior and converges faster in most cases

Learning Rate (η): The Most Important Hyperparameter

Too Large:

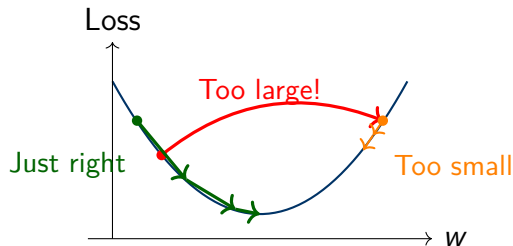
- ▶ Overshoots minimum
- ▶ Training diverges (loss $\rightarrow \infty$)

Too Small:

- ▶ Very slow training
- ▶ May get stuck

Just Right:

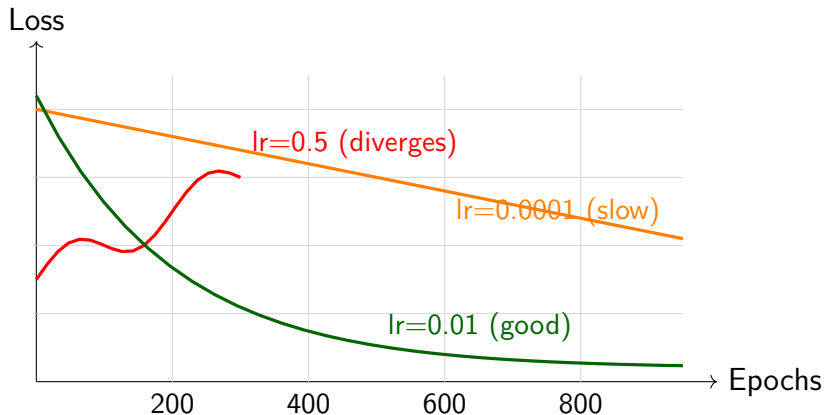
- ▶ Smooth convergence
- ▶ Reaches good solution



Typical values: SGD: 0.01–0.1 Adam: 0.001–0.0001

Effect of Learning Rate on Training

Training Loss vs Epochs for Different Learning Rates



A good learning rate shows smooth, steady decrease in loss

Why Does Weight Initialization Matter?

Problem:

- ▶ Too large → gradients explode
- ▶ Too small → gradients vanish

Solution: Scale weights by layer size

PyTorch:

```
nn.init.xavier_uniform_(  
    layer.weight)  
nn.init.kaiming_uniform_(  
    layer.weight)
```

Xavier: tanh/sigmoid

Kaiming: ReLU

Our Regression Problem

Goal: Train a neural network to learn:

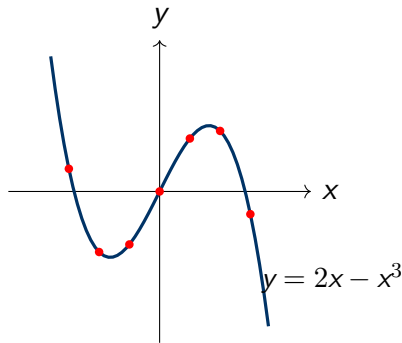
$$y = 2x - x^3$$

Data:

- ▶ 101 points from $x = -2$ to $x = 2$
- ▶ Input: x values
- ▶ Output: $y = 2x - x^3$

Network:

- ▶ 4 layers: $1 \rightarrow 10 \rightarrow 20 \rightarrow 10 \rightarrow 1$
- ▶ Activation: tanh



Red dots = training data points

Hands-on Exercise 1: Effect of Initialization

Goal: Compare random initialization vs Xavier initialization

Random Init (default)

```
nn.init.normal_(layer.weight)
```

→ Gradients may vanish/explode

Xavier Init (better)

```
nn.init.xavier_uniform_(  
    layer.weight)
```

→ Stable gradient flow

Run:

```
python exercise1_init.py
```

Observe:

- ▶ Training loss curves
- ▶ Final prediction quality
- ▶ Which converges better?

Hands-on Exercise 2: SGD vs Adam

Goal: Compare SGD and Adam optimizers

SGD – Simple, fixed learning rate

```
optim.SGD(model.parameters()  
          , lr=0.01)
```

Adam – Adaptive, uses momentum

```
optim.Adam(model.parameters  
            ), lr=0.001)
```

Run:

```
python exercise2_optimizers.py
```

Observe:

- ▶ Which converges faster?
- ▶ Final loss values
- ▶ Prediction accuracy

Hands-on Exercise 3: Learning Rate and Epochs

Goal: See how hyperparameters affect training

Learning Rate:

- ▶ $lr = 0.1$ – Too large?
- ▶ $lr = 0.01$ – Good?
- ▶ $lr = 0.001$ – Default for Adam
- ▶ $lr = 0.0001$ – Very slow?

Epochs:

- ▶ 100 – Too few?
- ▶ 500 – Better?
- ▶ 1000+ – Convergence?

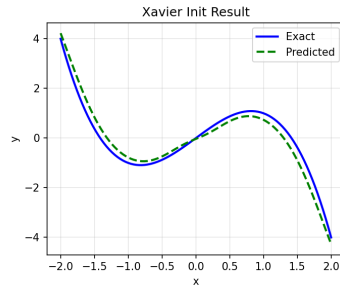
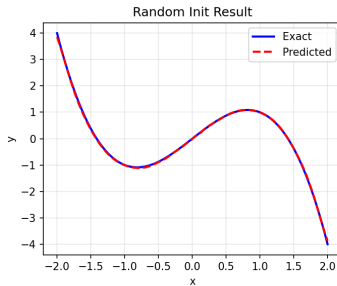
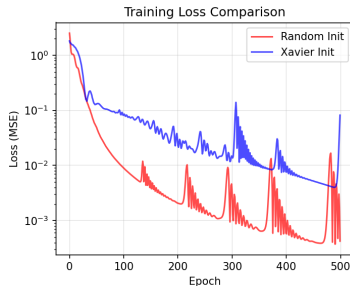
Run:

```
python exercise3_hyperparams.py
```

Observe:

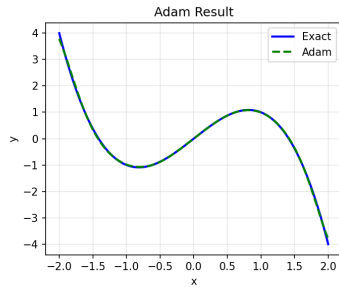
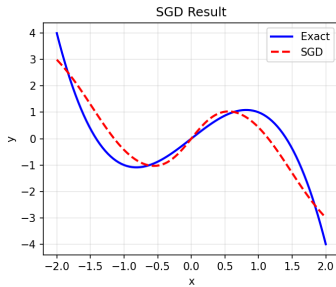
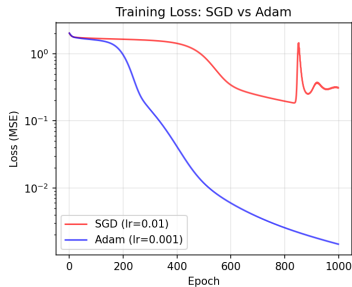
- ▶ Smooth vs diverging loss
- ▶ Convergence speed
- ▶ Full batch vs mini-batch

Exercise 1 Results: Initialization Comparison



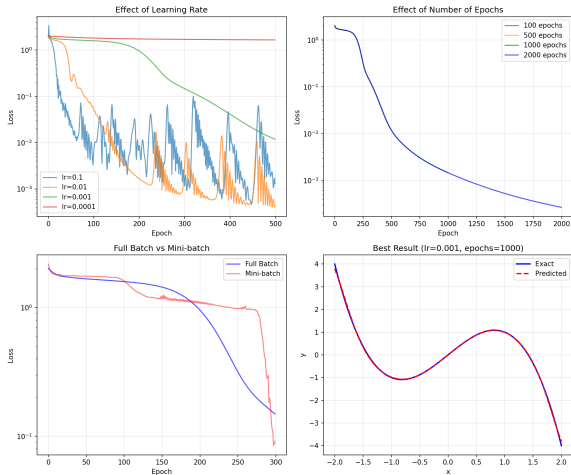
Xavier initialization helps maintain stable gradients across layers

Exercise 2 Results: SGD vs Adam



Adam converges faster and achieves lower loss than SGD

Exercise 3 Results: Hyperparameter Effects



Learning rate and epochs significantly affect training convergence

Summary: Key Takeaways

PyTorch Basics:

1. Create model with `nn.Module`
2. Define layers in `__init__`
3. Define forward pass in `forward`
4. Train with: `predict` $\rightarrow$ `loss` $\rightarrow$ `backward`

Optimizers:

- ▶ **SGD**: Simple, needs tuning
- ▶ **Adam**: Smart, usually works better

Hyperparameters:

- ▶ **Learning rate**: Start with 0.001 for Adam
- ▶ **Epochs**: Train until loss plateaus
- ▶ **Initialization**: Use Xavier for tanh

Best Practices:

1. Start with Adam optimizer
2. Use proper initialization
3. Monitor training loss
4. Adjust learning rate if needed

ॐ (३) क्रतो॑ स्मर॑ कृत॑ः स्मर॑ क्रतो॑ स्मर॑ कृत॑ः स्मर॑ ॥ १७ ॥

O my mind, remember all that has been done. Remember – remember all that has been done.

ॐ शा॒न्ति शा॒न्ति शा॒न्ति ॥